

Normalizzazione Rank-Based dei Timestamp per l'Esperimento del Problema 2

1 Problema riscontrato

L'esperimento del Problema 2 viene terminato dal kernel (OOM-kill) su tutti i dataset con timestamp in formato Unix epoch. Le cause identificate sono due, entrambe legate al fatto che il range temporale $T = t_{\max} - t_{\min}$ risulta enormemente più grande del numero di timestamp effettivamente distinti nel dataset.

1.1 Causa 1: vettore della soluzione alternativa

La classe `AlternativeSolution` alloca un vettore di $T - \mu + 1$ slot (uno per finestra temporale). Per dataset come `sx-stackoverflow`, con $T \approx 2.4 \times 10^8$ e $\mu \approx 1.2 \times 10^8$, questo vettore richiede circa 960 MB solo per i puntatori `nullptr`.

1.2 Causa 2: enumerazione esplicita di $W(v)$

La funzione `compute_W` enumera *ogni singolo intero* nell'unione degli intervalli $[\lambda - \mu, \lambda]$. Se il range normalizzato è dell'ordine di milioni, $|W(v)|$ può raggiungere decine di milioni per un singolo nodo, e la `SketchCache` memorizza uno sketch per ciascuno di questi valori.

2 Soluzione adottata: preprocessing di normalizzazione

Non è stato modificato alcun file dell'esperimento (`esperimento.cpp`, `esperimento_opt.cpp`, `plot_analysis.py`, `process_result.py`).

È stato creato un unico file aggiuntivo, `normalize_timestamps.cpp`, che implementa una **normalizzazione rank-based** dei timestamp come step di preprocessing.

2.1 Funzionamento

Dato un dataset con archi nel formato `u v t`:

1. **Passata 1:** raccoglie tutti i timestamp, li ordina, e rimuove i duplicati ottenendo la sequenza $t_0 < t_1 < \dots < t_{D-1}$ di D timestamp distinti.
2. **Passata 2:** riscrive il file sostituendo ogni timestamp t_i con il suo rango $i \in \{0, 1, \dots, D-1\}$.

Il file normalizzato viene salvato in `normalized_datasets/`.

2.2 Correttezza

La normalizzazione rank-based preserva tutte le proprietà rilevanti per l'esperimento:

- **L'ordinamento temporale** degli archi è invariante (il rango è una funzione monotona crescente del timestamp originale).

- Le **relazioni di vicinato** $N(v, t)$ restano identiche: l'insieme dei vicini di v al timestamp t non dipende dal valore numerico di t , ma solo da quali archi hanno quel timestamp.
- Gli **sketch MinHash** e i **RangeTree** producono esattamente gli stessi risultati, perché dipendono unicamente dall'insieme dei vicini a ciascun timestamp, non dal valore assoluto del timestamp.
- La **struttura LSH** e il **pruning** $W(v)$ operano sugli stessi insiemi di finestre temporali valide.

2.3 Effetto sulla compressione

Dataset	Range originale	Timestamp distinti	Compressione	μ consigliato
aves-sparrow	1	2	$1\times$	1
chess_year	8	9	$1\times$	4
CollegeMsg	16 736 181	58 911	$284\times$	10
email-Eu-core	69 459 254	207 880	$334\times$	5
soc-flickr	17 017 200	134	$127\,949\times$	10
sx-stackoverflow	239 705 551	52 423 237	$4.6\times$	–
wiki-talk	200 483 882	7 375 042	$27\times$	–

Tabella 1: Effetto della normalizzazione rank-based.

2.4 Calibrazione di μ

Dopo la normalizzazione, μ va espresso in termini di *numero di timestamp distinti consecutivi* nella finestra. La memoria occupata scala linearmente con μ : su una macchina con 8 GB di RAM (di cui ~ 4 GB disponibili), il limite pratico è circa $\mu \leq 50$ per CollegeMsg.

I dataset **sx-stackoverflow** e **wiki-talk** hanno rispettivamente 52M e 7M timestamp distinti: anche dopo la normalizzazione, la **TemporalRangeForest** da sola (con 2.6M e 1.1M nodi) eccede la memoria disponibile. Per questi dataset servirebbero almeno 32 GB di RAM.

3 File creati

File	Descrizione
<code>normalize_timestamps.cpp</code>	Normalizzatore rank-based dei timestamp
<code>normalized_datasets/*</code>	Dataset con timestamp normalizzati
<code>normalized_datasets/normalization_info.csv</code>	Metadati della normalizzazione
<code>run_all.sh</code>	Script bash per esecuzione completa

4 Riepilogo

1. **Nessun codice dell'esperimento è stato modificato.**
2. È stato aggiunto un **preprocessing** che normalizza i timestamp da valori Unix epoch a ranghi consecutivi $\{0, 1, \dots, D - 1\}$.
3. L'esperimento viene lanciato sui file normalizzati con valori di μ calibrati sul range compresso.